

BO2-05

Traçabilité des colonnes avec Neo4j

Rapport de réalisation - 8 octobre 2026
Branche : ahmed-ikbel | Version actualisée Neo4j

1. Objectif et résultat livré

Le DSO BO2-05 vise à remonter d'un indicateur aux données et opérations qui ont contribué à sa valeur. La solution réalisée calcule trois indicateurs sur les données synthétiques du projet, enregistre leurs dépendances dans Neo4j et permet de les inspecter dans Streamlit ou Neo4j Browser.

Cette version remplace le stockage actif SQLite de la première démonstration. Neo4j contient de vrais nœuds et relations ; les parcours amont sont obtenus par des requêtes Cypher. SQLite reste une archive conservée, importable sans suppression.

2. Architecture mise en place

Composant	Fonction
Python / pandas	Générer les données, appliquer le nettoyage 1.2.0 et calculer les indicateurs.
Pilote officiel Neo4j	Écrire le graphe en transaction et interroger les dépendances par Cypher.
Neo4j Community 5.26	Base de graphes exécutée localement dans Docker avec un volume persistant.
Streamlit	Sélectionner une exécution, un indicateur et un nœud ; consulter les preuves et exporter le JSON.
Neo4j Browser	Exécuter directement des requêtes et afficher les chemins du graphe.

Configuration locale

Streamlit : `http://127.0.0.1:8502` ; Neo4j Browser : `http://localhost:7474` ; connexion Bolt : `localhost:7687`. Les ports Neo4j sont exposés uniquement sur la machine locale. Les identifiants sont lus depuis `.env` ou les variables d'environnement ; le mot de passe n'est pas reproduit dans ce rapport.

La configuration Docker figure dans `compose.neo4j.yml`. Les données persistent dans un volume dédié ; arrêter le conteneur ne supprime pas ce volume.

Données, calculs et dépendances

3. Pipeline réellement tracé

Le dataset brut comprend 500 lignes, produites directement avec des imperfections par le générateur BO2-02 (graine 2026). La politique 1.2.0 nettoie les formats numériques et signale les cas non résolus. Les cinq champs métier sont age, income, spending_score, transaction_amount et num_transactions.

Le filtre retient uniquement les lignes sans exception sur ces cinq champs : **196 lignes retenues et 304 exclues des calculs**. Les données sources ne sont pas supprimées. Aucune jointure ou formule de chiffre d'affaires n'est inventée.

4. Indicateurs de démonstration

Indicateur	Formule sur lignes éligibles	Résultat
Somme des montants	sum(transaction_amount)	19 001,04
Revenu moyen	mean(income)	56 542,11
Total des transactions	sum(num_transactions)	3 833

Les résultats affichés sont arrondis à deux décimales ; les valeurs numériques non arrondies sont conservées. Les formules portent une version sémantique 1.0.0. Sans ligne éligible, le résultat est absent et affiché comme Non calculable.

5. Pourquoi le graphe possède plusieurs branches

Dépendance de valeur : seul income fournit les valeurs de la moyenne des revenus.

Dépendance de sélection : les cinq colonnes influencent la présence d'une ligne dans le calcul. Un âge invalide peut donc modifier le revenu moyen en excluant la ligne.

Chaque champ est représenté à plusieurs étapes : source, nettoyage et champ nettoyé. Les branches se rejoignent au filtre qualité avant le calcul. Le graphe complet conserve ces détails pour l'audit ; la disposition automatique de Neo4j Browser peut rendre l'affichage dense.

Lecture simplifiée du revenu moyen

Source	Préparation	Sélection	Indicateur
income et autres champs métier	Nettoyage selon 1.2.0	196 lignes valides sur les 5 champs	mean(income) = 56 542,11

Ce résultat décrit les lignes retenues, et non nécessairement la population complète. Le filtre peut introduire un biais de sélection ; il reste nécessaire de traiter les exceptions exclues.

Modèle Neo4j et persistance

6. Nœuds et relations natifs

Chaque exécution produit **24 nœuds métier et 34 relations de dépendance**. Neo4j ajoute un nœud d'exécution et 24 relations d'appartenance : au total, 25 nœuds et 58 relations par exécution dans la base.

Élément Neo4j	Propriétés et rôle
BIFlowRun	UUID, date UTC, métadonnées et copie JSON complète pour restituer fidèlement l'exécution.
BIFlowNode	Clé unique combinant exécution et identifiant local ; type, libellé, formule, compteur ou empreinte selon le nœud.
HAS_NODE	Relie une exécution à ses nœuds.
FLows_TO	Relie les étapes source vers résultat. La propriété relation distingue input, produces, value_dependency, row_selection, aggregation, etc.

7. Écriture et intégrité

Des contraintes d'unicité protègent l'identifiant de l'exécution et la clé des nœuds. L'exécution, les nœuds et les relations sont créés dans une transaction. Réimporter une exécution strictement identique ne la duplique pas ; une tentative avec le même identifiant mais un contenu différent est rejetée.

La version du schéma, la version des relations et les métadonnées de nettoyage sont conservées. Le manifeste inclut une copie de la politique, sa version et les empreintes SHA-256 de la politique et des données avant/après. Les lignes sources ne sont pas stockées dans le graphe.

8. Parcours et migration

Le parcours amont part du nœud indicateur sélectionné et recherche ses ancêtres par FLOWs_TO. Les requêtes sont limitées à l'exécution concernée, afin de ne pas mélanger les versions de plusieurs calculs. Streamlit récupère les nœuds et les relations de ce parcours depuis Neo4j.

Le module governance/migrate_lineage.py importe les anciennes exécutions SQLite en conservant leurs identifiants. Une ancienne exécution a été importée lors de la mise en place. La base historique n'a pas été supprimée. L'interface signale une panne Neo4j sans revenir silencieusement vers SQLite.

Fichiers principaux : governance/neo4j_store.py ; governance/lineage.py ; governance/migrate_lineage.py ; pages/4_BO2_Lineage.py ; compose.neo4j.yml ; tests/test_neo4j.py.

Tester le revenu moyen

9. Dans Streamlit

Ouvrir BO2-05, cliquer sur Calculer les indicateurs et enregistrer la traçabilité, sélectionner l'exécution puis Revenu moyen. Le résultat attendu sur le dataset actuel est **56 542,11 sur 196 lignes**. Inspecter le filtre pour vérifier les effectifs, puis un nœud de nettoyage pour retrouver sa règle.

10. Dans Neo4j Browser

La requête suivante sélectionne la dernière exécution et vérifie la valeur, le nombre de contributeurs et la formule. Elle ne nécessite pas de copier un UUID particulier.

Vérification du résultat

```
MATCH (r:BIFlowRun)
WITH r ORDER BY r.created_at DESC LIMIT 1
MATCH (k:BIFlowNode {run_id: r.id})
WHERE k.id = 'kpi:income_mean'
RETURN k.label AS indicateur, k.value AS valeur,
k.contributing_rows AS lignes, k.formula AS formule;
```

Pour afficher ses dépendances, exécuter la requête suivante, puis choisir l'onglet Graph. L'onglet Table affiche les mêmes chemins sous forme de lignes.

Parcours de traçabilité

```
MATCH (r:BIFlowRun)
WITH r ORDER BY r.created_at DESC LIMIT 1
MATCH p=(s:BIFlowNode)-[:FLOWS_TO*0..]->(k:BIFlowNode)
WHERE k.run_id = r.id AND k.id = 'kpi:income_mean'
RETURN p;
```

11. Vérifications et limites

20 tests ont réussi avec le test Neo4j réel activé. Le test d'intégration vérifie l'écriture, la relecture, l'import idempotent, la conformité du parcours amont et le refus d'un contenu contradictoire. Une vérification supplémentaire a validé la navigation Streamlit connectée à la base réelle.

La traçabilité reste instrumentée pour le nettoyage, le filtre et les trois indicateurs. Les DSO PII/anomalies, les entrepôts externes et OpenLineage ne sont pas encore intégrés au graphe. Aucun contrôle d'accès de production, historique infalsifiable ou archivage complet des sources n'est revendiqué.

Les rapports précédents décrivaient le prototype SQLite ; ce document décrit son évolution Neo4j.

Références techniques : [Neo4j Docker Compose](#) et [pilote Python officiel](#).